

Enter a valid parenthesized infix expression: (A+B-C^D)/(E*F)

Postfix expression: AB+CD^-EF*/

Process returned 0 (0x0) execution time : 21.769 s

Press any key to continue.

8) Write a program to convert a given valid parenthesized infix arithmetic expression to postfix expression. The expression consists of a single character operands and the binary operators + (plus), - (minus), * multiply and / (divide).

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
```

```
#define MAX 100
char stack[MAX];
int top = -1;
```

```
void push(char c)
{
    if (top == MAX - 1)
    {
        printf("Stack Overflow\n");
    }
    else
    {
        stack[++top] = c;
    }
}
```

```
char pop()
{
    if (top == -1)
    {
        printf("Stack Underflow\n");
        return -1;
    }
}
```



```

    else
    {
        return (stack[top--]);
    }
}

int precedence(char c)
{
    if (c == '+' || c == '-')
    {
        return 1;
    }
    else if (c == '*' || c == '/')
    {
        return 2;
    }
    else
    {
        return 0;
    }
}

```

```

void infixToPostfix(char infix[])
{

```

```

    char postfix[MAX];
    int i, j = 0;
    char ch;

```

```

    for (i = 0; i < strlen(infix); i++)
    {
        ch = infix[i];

```

```

        if (isalpha(ch) || isdigit(ch))

```



```

{
    postfix[j++] = ch;
}
else if (ch == '(')
{
    push(ch);
}
else if (ch == ')')
{
    while (stack[top] != '(' && (top != -1))
        postfix[j++] = pop();
    pop();
}
else
{
    while (precedence(stack[top]) >=
           precedence(ch))
        postfix[j++] = pop();
    push(ch);
}
}

while (top != -1)
    postfix[j++] = pop();

postfix[j] = '\0';
printf("Postfix expression: %s\n", postfix);
}

int main()
{
    char infix[MAX];
    printf("Enter a valid parenthesized infix expression: ");
    scanf("%s", infix);
}

```


infixToPostfix(infix);

return 0;

}

D/D Enter a valid parenthesized infix expression:

(A + B - C ^ D) / (E * F)

Postfix expression: AB+CD^=EF*/

6/10/20

